РО 1.3. Базовое администрирование Linux.

Лекция 10.
Командная строка Linux: Bash, справка, навигация, редакторы

Цель лекции

- Понять назначение терминала, командной оболочки и Bash;
- Научиться объяснять структуру команды: имя, опции, аргументы;
- Освоить логику навигации по файловой системе Linux;
- Понять, как пользоваться справкой, просмотром файлов и базовыми редакторами;

Список учебных вопросов

1. CLI и терминал: назначение командной строки для администратора Linux
2. Bash как командная оболочка: приглашение, команды, аргументы, опции
3. Абсолютный и относительный путь
4. Навигация по файловой системе: pwd, ls, cd
5. История команд и повторное выполнение команд
6. Справка по командам: man, --help, apropos
7. Просмотр файлов: cat, less, head, tail
8. Текстовые редакторы: nano, vi/vim на базовом уровне
9. Практическая аккуратность в терминале: рабочий каталог, проверка команды перед выполнением

1. CLI и терминал: назначение командной строки для администратора Linux

CLI — Command Line Interface, командный интерфейс управления системой.

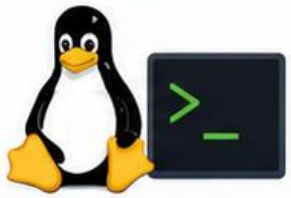
Терминал — программа или консоль, через которую пользователь взаимодействует с оболочкой.

Командная строка позволяет выполнять действия точно, повторяемо и удаленно.

Для администратора Linux терминал является основным рабочим инструментом.

Почему серверному администратору нужен CLI

- Сервер может не иметь графического интерфейса;
- Удаленное управление чаще выполняется через SSH;
- Команды удобно документировать в отчете и повторять на другом сервере;
- Командная строка показывает не только действие, но и диагностический результат;



CLI, терминал и Bash

Ваш интерфейс с Linux

Linux управляется через командную строку. Это быстро, мощно и позволяет полностью контролировать систему удалённо и локально.

1. CLI – Command Line Interface

CLI – это текстовый интерфейс для взаимодействия с системой с помощью команд.
Всё, что можно сделать в GUI, можно сделать в CLI (а иногда даже больше!).

2. Терминал

Терминал – это программа (эмулятор терминала), которая позволяет вводить команды и видеть их вывод.
Примеры терминалов: **GNOME Terminal, Konsole, xterm, Alacritty, Windows Terminal.**

3. Bash – командный интерпретатор

Bash (Bourne Again SHell) – это оболочка (shell), которая интерпретирует команды, введённые вами в терминале, и передаёт их ядру Linux для выполнения.

СТРУКТУРА ПРИГЛАШЕНИЯ (PROMPT)

```
user@server-01:~$
```

- `user` – пользователь
- `server-01` – имя коста
- `~` – текущая директория
- `$` – символ приглашения

После символа $ (обычный пользователь) или # (root) вводится команда.

user@server-01: ~

```
user@server-01:~$ whoami                          # кто я
user
user@server-01:~$ hostname                        # имя хоста
server-01
user@server-01:~$ pwd                             # текущая директория
/home/user
user@server-01:~$ ls -l
total 12
drwxr-xr-x 2 user user  4096 Jul  2 10:15 Documents
drwxr-xr-x 2 user user  4096 Jul  2 10:15 Downloads
-rw-r--r-- 1 user user   220 Jul  2 09:58 notes.txt
user@server-01:~$
```

ПОЧЕМУ CLI ВАЖЕН?

- **Работает везде** – Через SSH можно управлять сервером из любой точки мира.
- **Быстрее и легче** – CLI потребляет меньше ресурсов, чем графический интерфейс.
- **Автоматизация** – Скрипты и команды позволяют автоматизировать задачи.
- **Мощные возможности** – Многие системные задачи проще и гибче в CLI.
- **Стандарт в IT** – Администраторы и DevOps используют CLI ежедневно.

КАК ОТКРЫТЬ ТЕРМИНАЛ?

Linux	Меню приложений → Терминал или Ctrl + Alt + T
Windows	Windows Terminal / PowerShell или WSL (Windows Subsystem for Linux)
macOS	Приложение «Терминал» (Terminal.app) или iTerm2

ТИПЫ КОМАНД

- **Встроенные (builtin)** – cd, echo, read, type и др.
- **Внешние (executable)** – ls, cp, mv, grep, systemctl...
- **Скрипты и программы** – Любые исполняемые файлы или скрипты (bash, python и др.)

КАК ВЫПОЛНЯЕТСЯ КОМАНДА?

1. Вы вводите команду.
2. Bash ищет: встроенная команда? алиас? функция? внешняя программа?
3. Если это внешний файл – Bash запускает его как процесс.
4. Результат (вывод) возвращается в терминал.

ПОЛЕЗНЫЕ КОМБИНАЦИИ КЛАВИШ

Ctrl + C	прервать текущую команду
Ctrl + D	выход из текущей сессии (logout)
Ctrl + L	очистить экран
Ctrl + R	поиск в истории команд
Tab	автодополнение имён файлов и команд
↑ / ↓	навигация по истории команд

ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ

Хранят настройки системы и пользователя.

```
$ echo $HOME
/home/user
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Установить переменную: `export NAME=value`
Показать все переменные: `env` или `printenv`

БАЗОВЫЕ КОМАНДЫ

`ls`	список файлов и папок
`cd dir`	перейти в каталог
`pwd`	показать текущую директорию
`mkdir dir`	создать каталог
`rm file`	удалить файл
`cp src dst`	копировать файл или папку
`mv src dst`	переместить / переименовать
`cat file`	показать содержимое файла
`less file`	просмотр файла постранично
`man cmd`	справка по команде

НАВИГАЦИЯ ПО ФАЙЛОВОЙ СИСТЕМЕ

```
/ (корень)
├── bin
├── etc
├── home
│   └── user
│       ├── Documents
│       └── Downloads
└── var
```

`/`	корень файловой системы
`~`	домашняя директория пользователя
`.`	текущая директория
`..`	родительская директория
`-`	предыдущая директория (cd -)

ПАЙПЫ и ПЕРЕНАПРАВЛЕНИЕ

Пайп (|) – передаёт вывод одной команды на вход другой.

```
$ ls -l | grep ".txt"
-rw-r--r-- 1 user user 220 Jul  2 09:58 notes.txt
```

Перенаправление (>) – записать вывод в файл (перезаписать).

```
$ echo "Привет" > hello.txt
```

Добавление (>>) – добавить вывод в конец файла.

```
$ echo "Ещё строка" >> hello.txt
```

Чтение из файла (<)

```
$ sort < list.txt
```

ПОЛЕЗНЫЕ АЛИАСЫ (пример)

```
alias ll='ls -alF'        # подробный список
alias la='ls -A'          # без . и ..
alias l='ls -CF'          # цветной вывод
alias ..='cd ..'          # перейти вверх
alias ...='cd ../..'      # на 2 уровня вверх
alias cls='clear'         # очистить экран
```

Посмотреть все алиасы: `alias`
Добавить алиас: `echo "alias ll='ls -alF'" >> ~/.bashrc`

ПОЛЕЗНЫЕ СОВЕТЫ

- Используйте Tab – это ускоряет работу.
- История команд: `history`
- Проверяйте команды перед выполнением.
- Читайте справку: `man команда`
- Практикуйтесь ежедневно!

CLI – это как суперсила системного администратора. Чем лучше вы её знаете, тем больше возможностей у вас есть!

Терминал + Bash = полный контроль над системой.

Учитесь. Практикуйтесь. Автоматизируйте!

Терминал, оболочка и программа: кто за что отвечает

Терминал принимает ввод с клавиатуры и показывает вывод на экране.

Оболочка разбирает введенную строку и запускает команды.

Команда выполняет конкретную программу или встроенное действие оболочки.

Результат возвращается в терминал как текстовый вывод или сообщение об ошибке.

2. Bash как командная оболочка: приглашение, команды, аргументы, опции

Bash — Bourne Again Shell, популярная командная оболочка Linux.

Оболочка принимает команды, запускает программы и управляет окружением пользователя.

Приглашение показывает, что оболочка готова принять ввод.

Администратор читает приглашение, чтобы понимать пользователя, узел и текущий каталог.

Из чего состоит команда

Имя команды указывает, какую программу или встроенную функцию выполнить.

Опции изменяют поведение команды: обычно начинаются с **-** или **--**

Аргументы указывают объект действия: файл, каталог, имя службы, адрес.

Пример: **ls -la /etc** = команда **ls**, опции **-la**, аргумент **/etc**

Из чего состоит команда

Структура команды в Linux

Команда в Linux — это инструкция оболочке (shell) выполнить программу с нужными параметрами и обработать данные.

5 ОСНОВНЫХ ЧАСТЕЙ КОМАНДЫ

1. Команда (программа)	2. Опции (ключи)	3. Аргументы	4. Перенаправления	5. Конвейеры (pipes)
`ls`	`-l -a`	`/home/user`	`> output.txt`	`| grep "log"`
Что выполнить	Как выполнить	Над чем выполнить	Куда направить вывод	Передать вывод другой команде

ПРИМЕР ПОЛНОЙ КОМАНДЫ

```
ls -l /var/log > logs.txt | grep "error"
```

1	`ls`	команда (программа)
2	`-l`	опция: длинный список
3	`/var/log`	аргумент: каталог для просмотра
4	`> logs.txt`	перенаправление: сохранить вывод в файл
5	`| grep "error"`	конвейер: передать в команду grep

1 Команда (программа)

Это имя программы, которую нужно запустить. Обычно это системные утилиты или программы.

Примеры команд:

`ls`	просмотр содержимого каталога
`cd`	смена каталога
`cp`	копирование файлов
`mv`	перемещение/переименование
`rm`	удаление файлов
`cat`	вывод содержимого файла
`grep`	поиск текста

2 Опции (ключи)

Опции изменяют поведение команды. Начинаются с одного или двух дефисов.

Виды опций:

- `-a` короткая опция
- `--all` длинная опция
- `-l -a` несколько коротких опций (объединяются)

Примеры:

`ls -l`	длинный список
`cp -r`	копировать рекурсивно
`grep -i "text"`	без учёта регистра

3 Аргументы

Аргументы — это объекты (файлы, каталоги, имена, числа и т.д.), над которыми выполняется команда.

Примеры аргументов:

- файлы: `file.txt`, `/etc/passwd`
- каталоги: `/home/user`, `/var/log`
- значения: `10`, `192.168.1.1`, `user`
- шаблоны: `*.txt`, `?a?.log`

Пример:

`cat /etc/hosts` → файл /etc/hosts

4 Перенаправления

Перенаправления позволяют работать с вводом и выводом команды.

Основные операторы:

`>`	перенаправить вывод в файл (перезаписать)
`>>`	добавить вывод в конец файла
`<`	взять ввод из файла
`2>`	перенаправить ошибки в файл
`&>`	перенаправить всё (stdout и stderr)

Пример:

`ls -l > list.txt` → сохранить вывод в файл

5 Конвейеры (pipes)

Конвейер `|` передаёт вывод одной команды на ввод другой.

Пример:

```
ps aux | grep nginx
```

вывод команды ps — используется как ввод для grep

ПРАВИЛА НАПИСАНИЯ КОМАНД

- Между частями команды ставятся пробелы.
- Порядок важен: команда → опции → аргументы → перенаправления → конвейеры.
- Регистр букв имеет значение.
- Если в аргументе есть пробелы — используйте кавычки: `"текст с пробелами"`
- Tab — автодополнение команд и путей.

ПОРЯДОК ОБРАБОТКИ КОМАНДЫ ОБОЛОЧКОЙ

1. Чтение команды пользователя
2. Разбор на части (лексический анализ)
3. Обработка перенаправлений
4. Настройка конвейеров
5. Запуск команд на выполнение

ПОЛЕЗНЫЙ СОВЕТ

Используйте man <команда> или <команда> --help для просмотра справки по команде и её опциям.

```
man ls    или    ls --help
```

Помните: Чем лучше вы понимаете структуру команд, тем эффективнее работаете в терминале!

Практикуйтесь каждый день!

Что означает приглашение командной строки

Имя пользователя помогает понять, от чьего имени выполняется команда.

Имя узла показывает, на каком сервере открыт сеанс.

Текущий каталог определяет контекст относительных путей.

Символ **$** обычно означает обычного пользователя, **#** — административную оболочку root.

Проверка текущего каталога

Задача: понять контекст, в котором будут выполняться относительные пути.

Команда: **pwd**

Пример использования: **pwd**

Пример результата: **/home/student**

Как читать результат: сейчас рабочий каталог пользователя student; относительные пути будут строиться от него.

```
user@srv-01:~$ pwd
/home/user
user@srv-01:~$
```

3. Абсолютный и относительный путь

Путь — способ указать расположение файла или каталога в файловой системе.

Абсолютный путь начинается от корня / и не зависит от текущего каталога.

Относительный путь начинается от текущего рабочего каталога.

Понимание путей защищает от изменения не того файла.

Примеры путей

/etc/ssh/sshd_config — абсолютный путь к конфигурации SSH.

docs/report.txt — относительный путь от текущего каталога.

. означает текущий каталог

.. означает каталог уровнем выше

~ означает домашний каталог текущего пользователя

Переход по абсолютному пути

Задача: перейти в известный системный каталог независимо от текущего места.

Команда: **cd**

Пример использования: **cd /etc**

Пример результата: pwd выводит /etc

```
user@srv-01:~$ cd /etc
user@srv-01:/etc$ pwd
/etc
user@srv-01:/etc$
```

Переход по относительному пути

Задача: перейти в каталог относительно текущего положения.

Команда: **cd имя_каталога**

Пример использования: **cd ssh**

Пример результата: pwd выводит /etc/ssh

Результат зависит от того, где пользователь был до команды.

```
user@srv-01:/etc$ cd ssh
user@srv-01:/etc/ssh$ pwd
/etc/ssh
user@srv-01:/etc/ssh$
```

4. Навигация по файловой системе: pwd, ls, cd

- Навигация — это умение ориентироваться в дереве каталогов Linux;
- pwd показывает текущий рабочий каталог;
- **ls** показывает содержимое каталога;
- **cd** меняет текущий рабочий каталог;
- Эти команды нужны перед любыми действиями с файлами и конфигурацией;

Просмотр содержимого каталога

Задача: увидеть объекты в текущем или указанном каталоге.

Команда: **ls**

Пример использования: **ls /usr**

Как читать результат: каталог /usr содержит программы, библиотеки и общие ресурсы.

```
user@srv-01:~$ ls /usr
bin  games  include  lib  lib64  libexec  local  sbin  share  src
user@srv-01:~$
```

Подробный список файлов

Задача администратора: увидеть права, владельца, размер и время изменения.

Команда: **ls -la путь**

Пример использования: **ls -la /etc**

Пример результата: строки с правами, владельцами и именами файлов.

Длинный формат помогает оценить тип объекта и права доступа.

```
user@srv-01:~$ ls -la /usr
total 72
drwxr-xr-x 12 root root  4096 May 14 15:43 .
drwxr-xr-x 18 root root  4096 May 14 15:49 ..
drwxr-xr-x  2 root root 20480 Jul  2 14:36 bin
drwxr-xr-x  2 root root  4096 Jan  2 17:35 games
drwxr-xr-x  4 root root  4096 Jul  2 14:35 include
drwxr-xr-x 50 root root  4096 Jul  2 14:36 lib
drwxr-xr-x  2 root root  4096 May 14 15:48 lib64
drwxr-xr-x  9 root root  4096 Jul  2 14:35 libexec
drwxr-xr-x 11 root root  4096 May 14 15:43 local
drwxr-xr-x  2 root root 12288 Jul  2 14:36 sbin
drwxr-xr-x 97 root root  4096 Jul  2 14:36 share
drwxr-xr-x  2 root root  4096 Jan  2 17:35 src
user@srv-01:~$
```

Возврат в домашний каталог

Задача: быстро вернуться в безопасное рабочее место пользователя.

Команда: **cd**

Пример использования: **cd**

Пример результата: pwd выводит /home/student

Без аргументов **cd** возвращает в домашний каталог текущего пользователя.

НАВИГАЦИЯ ПО ФАЙЛОВОЙ СИСТЕМЕ LINUX

pwd, ls, cd — базовые команды для работы с каталогами и файлами

В Linux всё есть файл.
Файловая система имеет древовидную структуру, начинающуюся с корня: /

pwd — показать текущий каталог

Выводит полный путь к каталогу, в котором вы находитесь.

```
user@server:~$ pwd
/home/user
```

- ~ (тильда) — ваш домашний каталог (/home/user)
- / — корень файловой системы
- Путь может быть абсолютным: /home/user/docs или относительным (см. cd).

ls — список файлов и каталогов

Показывает содержимое каталога.

```
user@server:~$ ls
bin   Documents   Downloads   file.txt   Music   Pictures
```

Полезные опции ls

`ls`	список файлов
`ls -l`	длинный формат (права, владелец, размер, дата)
`ls -a`	показать скрытые файлы (начинаются с .)
`ls -la`	длинный формат + скрытые файлы
`ls -h`	читаемые размеры (K, M, G)
`ls -lh`	длинный формат + читаемые размеры
`ls -R`	рекурсивно (в подкаталогах)
`ls -t`	сортировка по времени изменения (новые сверху)

cd — сменить каталог

Позволяет переходить между каталогами.

```
user@server:~$ cd Documents
user@server:~/Documents$ pwd
/home/user/Documents
```

Способы перехода

`cd Documents`	перейти в подкаталог Documents
`cd ..`	подняться на уровень выше
`cd /`	перейти в корень файловой системы
`cd ~` или `cd`	перейти в домашний каталог
`cd -`	вернуться в предыдущий каталог

Важно! Файлы и каталоги чувствительны к регистру: `File.txt` и `file.txt` — разные объекты.

ДЕРЕВООБРАЗНАЯ СТРУКТУРА

- / — корневой каталог системы
  - bin — основные команды
  - etc — конфигурации системы
  - home — домашние каталоги пользователей
    - user — домашний каталог пользователя
      - Documents
      - Downloads
      - Pictures
      - ...
  - var — переменные данные (логи, кеши и т.д.)
  - tmp — временные файлы
  - ...

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ ls

ls

```
user@server:~$ ls
Desktop   Documents   file.txt   Music   Pictures
```

ls -a

```
user@server:~$ ls -a
.    .    .bashrc   .config   Desktop   Documents
file.txt   .local   Music   Pictures   .profile
```

ls -l

```
user@server:~$ ls -l
total 20
drwxr-xr-x 2 user user 4096 мая 12 10:15 Desktop
drwxr-xr-x 2 user user 4096 мая 12 10:15 Documents
-rw-r--r-- 1 user user 1234 мая 12 09:50 file.txt
drwxr-xr-x 2 user user 4096 мая 12 10:15 Music
drwxr-xr-x 2 user user 4096 мая 12 10:15 Pictures
```

ls -lh

```
user@server:~$ ls -lh
total 20K
drwxr-xr-x 2 user user 4,0K мая 12 10:15 Desktop
drwxr-xr-x 2 user user 4,0K мая 12 10:15 Documents
-rw-r--r-- 1 user user 1,3K мая 12 09:50 file.txt
drwxr-xr-x 2 user user 4,0K мая 12 10:15 Music
drwxr-xr-x 2 user user 4,0K мая 12 10:15 Pictures
```

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ cd

cd Documents — переход в подкаталог

```
user@server:~$ cd Documents
user@server:~/Documents$ pwd
/home/user/Documents
```

cd .. — подняться выше

```
user@server:~/Documents$ cd ..
user@server:~$ pwd
/home/user
```

cd / — корень

```
user@server:~$ cd /
user@server:/$ pwd
/
```

cd ~ или cd — домашний каталог

```
user@server:/$ cd ~
user@server:~$ pwd
/home/user
```

cd - — вернуться назад

```
user@server:~$ cd Documents
user@server:~/Documents$ cd -
/home/user
```

СКРЫТЫЕ ФАЙЛЫ

Файлы и каталоги, имена которых начинаются с точки ., скрыты по умолчанию.

Пример:

```
user@server:~$ ls -a
.    ..    .bashrc   .cache   .config   .ssh
```

ТИПЫ ПУТЕЙ

Абсолютный путь
Всегда начинается с / (корня).
Пример:
`/home/user/Documents/file.txt`

Относительный путь
Отсчитывается от текущего каталога.
Пример:
`Documents/file.txt`

ПОЛЕЗНЫЕ СОВЕТЫ

- Используйте Tab для автодополнения имён файлов и каталогов.
- Команда history покажет историю введённых команд.
- Команда clear очищает экран терминала.
- Изучайте man <команда> для подробной справки.

ПАМЯТКА

pwd	показать, где я
ls	что здесь находится
cd	куда я перехожу

5. История команд и повторное выполнение команд

История команд хранит ранее введенные команды текущего пользователя.

Стрелки вверх и вниз помогают быстро повторить или изменить команду.

history показывает список команд с номерами.

История экономит время, но опасна при повторении команд удаления или изменения прав.

Просмотр истории команд

Задача администратора: найти ранее выполненную команду и повторить ее осознанно.

Команда: **history**

Пример использования: **history**

Пример результата: список команд с номерами.

```
user@srv-01:~$ history
    1  ping 8.8.8.8
    2  clear
    3  whoami
    4  id
    5  cat /etc/os-release
    6  ip a
    7  sudo hostnamectl set-hostname srv-01
    8  exec bash
    9  sudo hostnamectl set-hostname debian
   10  exec bash
   11  sudo hostnamectl set-hostname srv-01
   12  exec bash
   13  ip a
```

Безопасное использование истории

Не нажимать Enter автоматически после выбора команды из истории.

Проверять путь в командах rm, chmod, chown, mv.

Не повторять sudo-команду, если непонятно, зачем она выполнялась.

История помогает учиться, но не заменяет понимание действия.

6. Справка по командам: man, --help, apropos

Администратор не обязан помнить все параметры команд.

Важно уметь быстро найти назначение команды и правильный синтаксис.

man открывает подробное руководство

--help показывает краткую справку

apropos ищет команды по ключевому слову в описании.

Открытие руководства man

Задача: изучить назначение команды и ее параметры.
Команда: **man команда**
Пример использования: **man ls**
Пример результата: страница руководства
Как читать результат: man подходит для подробного изучения команды;
Выход выполняется клавишей **q**

```
user@srv-01:~$ man ls
              slash (-p), file-type (--file-type), classify (-F)

       -i, --inode
              print the index number of each file

       -I, --ignore=PATTERN
              do not list implied entries matching shell PATTERN

       -k, --kibibytes
              default to 1024-byte blocks for file system usage; used only with
              -s and per directory totals

       -l     use a long listing format
```

Краткая справка по команде

Задача: быстро уточнить основные опции.
Команда: **команда --help**
Пример использования: **ls --help**
Пример результата: список опций ls
Формат --help удобен, когда нужно быстро вспомнить ключ.

```
user@srv-01:~$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
      --author               with -l, print the author of each file
  -b, --escape               print C-style escapes for nongraphic characters
      --block-size=SIZE      with -l, scale sizes by SIZE when printing them;
                             e.g., '--block-size=M'; see SIZE format below
```

Поиск команды по назначению

Задача: найти подходящую команду, если известно действие.
Команда: **apropos слово**
Пример использования: **apropos copy**
Пример результата: список команд, связанных с копированием.

```
user@srv-01:~$ apropos copy
cp (1)                   - copy files and directories
cpio (1)                 - copy files to and from archives
dd (1)                   - convert and copy a file
debconf-copydb (1)       - copy a debconf database
install (1)              - copy files and set attributes
scp (1)                  - OpenSSH secure file copy
ssh-copy-id (1)          - use locally available keys to authorise
string_copying (7)       - copying strings and character sequences
```

7. Просмотр файлов: cat, less, head, tail

Просмотр файла — частая задача при чтении конфигурации, журналов и отчетов.
cat удобен для маленьких файлов
less удобен для длинных файлов и журналов
head показывает начало файла, **tail** — конец файла
Выбор команды зависит от размера файла и цели просмотра.

Просмотр небольшого файла

Задача администратора: быстро вывести короткий текстовый файл

Команда: **cat файл**

Пример использования: **cat /etc/os-release**

Пример результата: строки с названием и версией ОС

cat не подходит для больших журналов, потому что весь вывод уйдет в терминал.

Постраничный просмотр файла

Задача: читать длинный файл без потери ориентации.
Команда: **less файл**
Пример использования: **less /etc/passwd**
Пример результата: файл открыт в просмотрщике less
Как читать результат: можно листать, искать через **/слово** и выйти клавишей **q**

Просмотр начала файла

Задача администратора: посмотреть первые строки конфигурации или отчета.
Команда: **head -n число файл**
Пример использования: **head -n 20 /etc/passwd**
Пример результата: первые 20 строк файла.
head полезен, когда структура важна в начале файла

Просмотр конца файла

Задача: увидеть последние строки, особенно в журналах.

Команда: **tail -n число файл**

Пример использования: **tail -n 30 /var/log/syslog**

Пример результата: последние 30 строк файла

tail помогает быстро увидеть последние события



ПРОСМОТР ФАЙЛОВ В LINUX

cat, less, head, tail — основные команды для чтения содержимого файлов

ЗАЧЕМ СМОТРЕТЬ ФАЙЛЫ В ТЕРМИНАЛЕ?

- ✓ Просмотр конфигурационных файлов
- ✓ Анализ логов и отчётов
- ✓ Быстрый доступ к данным без открытия редактора

1. cat — вывести весь файл

Выводит всё содержимое файла в терминал.

```
$ cat file.txt
Первая строка
Вторая строка
Третья строка
Четвёртая строка
...
```

- ➕ Просто и быстро.
- ➖ Неудобно для больших файлов (выводится всё сразу).

Примеры

```
$ cat /etc/hosts          # просмотр файла hosts
$ cat file.txt > copy.txt # скопировать в другой файл
$ cat -n file.txt         # показать с номерами строк
```

2. less — постраничный просмотр

Показывает файл постранично, удобно для больших файлов.

```
$ less /var/log/syslog
(содержимое файла)
--Нажмите q для выхода--
```

Управление в less

Пробел или Page Down	следующая страница
b или Page Up	предыдущая страница
↑ / ↓	построчно вверх/вниз
/текст	поиск вперёд
?текст	поиск назад
n	следующий результат поиска
N	предыдущий результат поиска
q	выход

Примеры

```
$ less largefile.log      # открыть большой файл
$ less +20 file.txt       # открыть с 20-й строки
```

3. head — начало файла

Выводит первые строки файла.

```
$ head file.txt
Первая строка
Вторая строка
Третья строка
Четвёртая строка
Пятая строка
```

Опции

-n 10	показать первые 10 строк
-n -10	показать все кроме последних 10 строк
-c 100	показать первые 100 байт

Примеры

```
$ head file.txt           # первые 10 строк (по умолчанию)
$ head -n 20 file.txt     # первые 20 строк
$ head -c 200 file.txt    # первые 200 байт
```

4. tail — конец файла

Выводит последние строки файла.

```
$ tail file.txt
...
Предпоследняя строка
Последняя строка
```

Опции

-n 10	показать последние 10 строк
-n +10	показать с 10-й строки до конца
-f	следить за изменениями файла (режим реального времени)

Примеры

```
$ tail file.txt           # последние 10 строк (по умолчанию)
$ tail -n 50 file.txt     # последние 50 строк
$ tail -f /var/log/syslog # следить за логом в реальном времени
```

СРАВНЕНИЕ КОМАНД

Команда	Что показывает	Для чего подходит
cat	Весь файл целиком	Маленькие файлы, быстрый вывод
less	Файл постранично	Большие файлы, удобный просмотр
head	Начало файла	Проверить заголовок, первые строки
tail	Конец файла	Проверить последние строки, логи

РАБОТА С БОЛЬШИМИ ЛОГАМИ

Комбинируйте команды для удобного анализа логов.

```
$ less /var/log/syslog                  # постраничный просмотр
$ tail -f /var/log/syslog               # следить за логом в реальном времени
$ tail -n 100 /var/log/syslog | less    # последние 100 строк через less
```

ПОЛЕЗНЫЕ СОВЕТЫ

- ✓ Для больших файлов используйте `less` вместо `cat`.
- ✓ Нажмите `q` в `less` для выхода.
- ✓ Для логов используйте `tail -f`.
- ✓ Команды `head` и `tail` не загружают весь файл в память.
- ✓ Можно читать файлы от `root`: `sudo cat /etc/shadow`

ПРИМЕР СОДЕРЖИМОГО FILE.TXT

```
1  Первая строка
2  Вторая строка
3  Третья строка
4  Четвёртая строка
5  Пятая строка
   ...
96 Предпоследняя строка
97 Последняя строка
```

Основная идея: `cat` — быстро, `less` — удобно, `head` — начало, `tail` — конец.
Эти команды помогут вам быстро находить нужную информацию в файлах!

Практика — лучший способ запомнить команды!

8. Текстовые редакторы: nano, vi/vim на базовом уровне

Редактор нужен для изменения конфигурационных файлов и создания заметок.

nano проще для начинающих и показывает подсказки клавиш.

vi/vim часто доступен даже в минимальной серверной установке.

Перед изменением системного файла нужно понимать, как сохранить и выйти.

Редактирование файла в nano

Задача: создать или изменить учебный текстовый файл.

Команда: **nano файл**

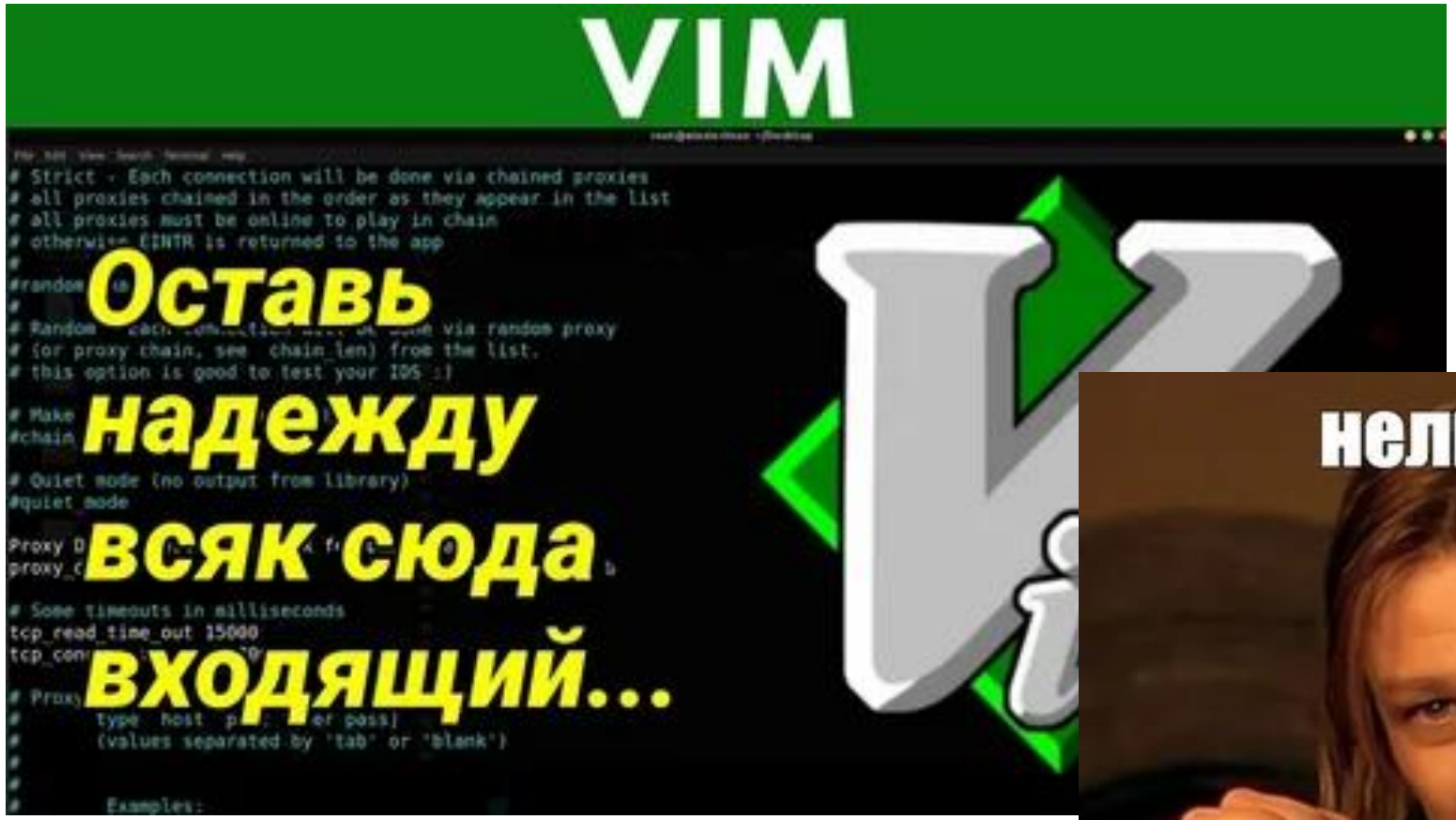
Пример использования: **nano notes.txt**

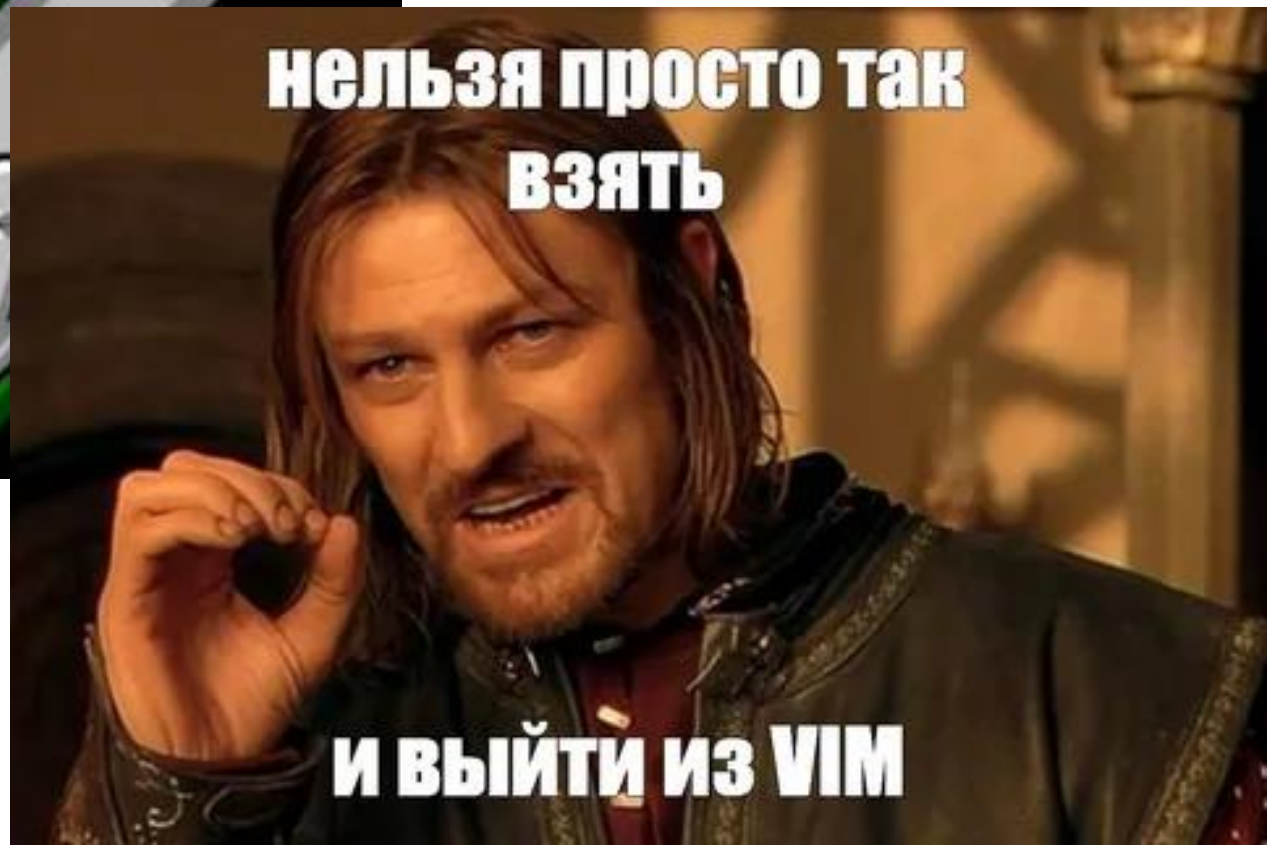
Пример результата: открывается редактор nano

Ctrl+O сохраняет

Ctrl+X закрывает редактор

Открытие файла в vim





Открытие файла в vim

Задача: Выйти из редактора, который часто есть на сервере.

Команда: **vim файл**

Пример использования: **vim notes.txt**

Пример результата: открывается редактор vim

Как пользоваться:

i -включает ввод,

Esc затем **:wq -**сохраняет и выходит,

:q! -выходит без сохранения

ТЕКСТОВЫЕ РЕДАКТОРЫ В LINUX: nano, vi/vim

Два мощных инструмента для создания и редактирования текстовых файлов в терминале

Когда что использовать?

- nano — просто и понятно, идеально для новичков
- vi/vim — мощный и быстрый, стандарт на серверах и в DevOps

NANO — ПРОСТОЙ И ПОНЯТНЫЙ РЕДАКТОР

Запуск

```
$ nano filename.txt
```

Создаст новый файл, если его не существует.

Интерфейс nano

```
GNU nano 7.2          filename.txt          Modified
 1 Первая строка
 2 Вторая строка
 3 Третья строка
 4

^G Help   ^O Write Out ^W Where Is  ^K Cut    ^T Execute
^X Exit   ^R Read File ^\ Replace   ^U Paste  ^J Justify
```

Основные команды nano

Ctrl + O	сохранить файл (Write Out)
Ctrl + X	выйти из nano
Ctrl + R	открыть файл для чтения
Ctrl + W	поиск текста
Ctrl + \	замена текста
Ctrl + K	вырезать строку
Ctrl + U	вставить строку
Ctrl + J	выровнять текст по ширине
Ctrl + C	показать текущую строку/позицию
Ctrl + _	перейти к строке

Советы по nano

- Подсказки по командам всегда внизу экрана.
- Используйте стрелки для перемещения.
- Очень удобно для быстрых правок конфигов.

VI / VIM — МОЩНЫЙ И БЫСТРЫЙ РЕДАКТОР

Запуск

```
$ vi filename.txt
или
$ vim filename.txt
```

Создаст новый файл, если его не существует.

Режимы работы vi/vim

Командный (по умолчанию)	⇄	Режим вставки (редактирование)	⇄	Режим последней строки (команды сохранения)
Навигация, копирование, удаление и др.		Ввод и редактирование текста как обычно		Сохранение, выход, поиск, замена и др.

Переход в режим вставки

i	вставить перед курсором
a	вставить после курсора
o	новая строка ниже
O	новая строка выше
I	в начало строки
A	в конец строки

Интерфейс vi/vim

```
filename.txt                1,1           All
Первая строка
Вторая строка
Третья строка


-- COMMAND --
```

Навигация в командном режиме

h	влево
j	вниз
k	вверх
l	вправо
w	слово вправо
b	слово влево
0	в начало строки
$	в конец строки
gg	в начало файла
G	в конец файла
:n	перейти к строке n

Основные команды в командном режиме

x	удалить символ под курсором
dd	удалить текущую строку
yy	скопировать строку
p	вставить после курсора
u	отменить действие
/текст	поиск вперед
?текст	поиск назад
n	повторить поиск
N	повторить поиск назад

Режим последней строки (двоеточие :)

:w	сохранить файл
:q	выйти
:wq	сохранить и выйти
:q!	выйти без сохранения
:set nu	показать номера строк
:set nonu	скрыть номера строк
:%s/старое/новое/g	замена по всему файлу
:h команда	справка по командам

СРАВНЕНИЕ NANO И VI/VIM

Критерий	nano	vi/vim
Простота	Очень простой, легко освоить	Требует времени на изучение
Интерфейс	Подсказки внизу экрана	Минималистичный интерфейс
Режимы работы	Один основной режим	Несколько режимов
Скорость работы	Хорошая	Очень высокая (особенно в vim)
Подходит для	Новичков, быстрых правок	Опытных пользователей, админов, разработчиков
Установлен по умолчанию	Есть не во всех дистрибутивах	Есть практически везде

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

$ nano /etc/hosts	быстро отредактировать hosts
$ nano script.sh	создать и редактировать скрипт
$ vi /etc/nginx/nginx.conf	редактировать конфигурации на сервере
$ vim file.txt	удобно работать с большими файлами

Советы для nano

- Сохраняйте часто: Ctrl + O
- Выход: Ctrl + X
- Используйте поиск: Ctrl + W
- nano отлично подходит для новичков.

Советы для vi/vim

- Учите базовые команды движения (hjkl).
- Практикуйте в безопасном файле.
- vim ускоряет работу в терминале в разы.
- Нажмите :help для встроенной справки.

ПАМЯТКА: КАК СОХРАНИТЬ И ВЫЙТИ

nano	vi/vim
1. Ctrl + O (сохранить)	1. :w (сохранить)
2. Enter (подтвердить имя файла)	2. :q (выйти)
3. Ctrl + X (выйти)	3. :wq (сохранить и выйти)

Освойте nano для простых задач и vi/vim для профессионального уровня!

Практика — лучший способ стать мастером Linux!

9. Практическая аккуратность в терминале: рабочий каталог, проверка команды перед выполнением

- Команда выполняется в конкретном контексте: пользователь, сервер, каталог, права;
- Перед изменением файлов нужно проверить pwd и ls;
- Перед использованием sudo нужно понимать последствия;
- Опасные команды сначала отрабатываются на учебных файлах;

Мини-чек-лист перед выполнением команды

- Я на нужном сервере?
- Я работаю нужным пользователем?
- Я нахожусь в правильном каталоге?
- Я понимаю, что изменит команда?
- Я смогу проверить результат после выполнения?

Итоги лекции

- Терминал дает доступ к оболочке, а оболочка выполняет команды;
- Bash разбирает команду на имя, опции и аргументы;
- Навигация строится на текущем каталоге и понимании путей;
- Справка и просмотр файлов являются частью профессиональной работы администратора;
- Аккуратность в терминале важнее скорости набора команд;

Контрольные вопросы

1. Чем терминал отличается от командной оболочки?
2. Что такое Bash?
3. Из каких частей состоит команда ls -la /etc?
4. Чем абсолютный путь отличается от относительного?
5. Для чего нужна команда pwd?
6. Когда лучше использовать less вместо cat?
7. Как получить справку по команде?
8. Что нужно проверить перед выполнением опасной команды?